Contents

Top 10 Self-Hosted Risks 2026

Free Guide by ClawGuru

> **This guide is free. No signup required.**
> Share it with your team, use it in your security reviews, and stay safe.

Executive Summary

Self-hosting gives you control, but it also comes with risks. In 2026, the most common self-hosted security failures are not sophisticated attacks — they are basic misconfigurations that any attacker can exploit.

This guide covers the **Top 10 Self-Hosted Risks** we see in production, with concrete fixes you can implement today.

Estimated reading time: 10 minutes
Difficulty: Beginner to Intermediate
Impact: Critical for any self-hosted infrastructure

Risk #1: Exposed Docker Socket

Severity: □ Critical
Common in: Dev environments, CI/CD pipelines, monitoring stacks

The Problem

The Docker socket (`/var/run/docker.sock`) gives full container control to anyone with access. If you expose this socket to the internet or bind-mount it into a container, an attacker can: - Create new containers with privileged access - Mount host filesystems - Escape container isolation completely

Real-World Impact

- **2025:** A dev team exposed the Docker socket via a reverse proxy for "convenience." Attackers spun up crypto-mining containers within 24 hours.
- **2024:** CI/CD pipeline with exposed socket allowed supply chain injection.

The Fix

```
# NEVER expose Docker socket to the internet
# DO NOT bind-mount /var/run/docker.sock into containers unless absolutely necessary

# If you must use it (e.g., Portainer, Traefik):
# 1. Restrict to localhost only
docker run -v /var/run/docker.sock:/var/run/docker.sock \
  -p 127.0.0.1:9000:9000 \
  portainer/portainer

# 2. Use a reverse proxy with authentication
# 3. Monitor socket access logs
```

Prevention Checklist

- ☐ Docker socket never exposed to 0.0.0.0
- ☐ All containers using socket run with least privileges
- ☐ Socket access logged and monitored
- ☐ Alternative: Use Docker API over TLS with client certificates

---

Risk #2: Default SSH Keys

Severity: Critical
Common in: Cloud instances, VPS, homelabs

The Problem

Default SSH keys (e.g., `authorized_keys` from cloud provider templates, shared keys across servers) allow anyone who knows the key to access your infrastructure.

Real-World Impact

- **2025:** Attacker scanned for default AWS EC2 SSH keys, compromised 200+ servers.
- **2024:** Homelab user reused default key from YouTube tutorial — full compromise.

The Fix

```
# Generate unique SSH key per server
ssh-keygen -t ed25519 -f ~/.ssh/my-server-key -C "my-server"

# Disable password authentication
sudo nano /etc/ssh/sshd_config
# Set: PasswordAuthentication no
# Set: PubkeyAuthentication yes
```

```
# Use key-based auth only
ssh -i ~/.ssh/my-server-key user@server
```

Prevention Checklist

- ☐ Unique SSH key per server
- ☐ Password authentication disabled
- ☐ Root login disabled
- ☐ SSH keys rotated every 90 days
- ☐ SSH access logged (auth.log, fail2ban)

Risk #3: Unauthenticated Redis/MongoDB

Severity: Critical
Common in: Development databases, caching layers

The Problem

Redis and MongoDB often ship with no authentication by default. If exposed to the internet, anyone can read, write, or delete data.

Real-World Impact

- **2025:** Unauthenticated Redis instance used as DDoS amplifier.
- **2024:** MongoDB with no auth — attacker deleted all databases and left ransom note.

The Fix

```
# Redis: Enable password auth
redis-cli
CONFIG SET requirepass "your-strong-password-here"
CONFIG SET masterauth "your-strong-password-here"

# MongoDB: Enable authentication
# In mongod.conf:
security:
  authorization: enabled
# Create admin user:
mongo
use admin
db.createUser({
  user: "admin",
  pwd: "strong-password",
  roles: [ { role: "root", db: "admin" } ]
})
```

Prevention Checklist

- ☐ Redis: `requirepass` set
- ☐ MongoDB: Authentication enabled
- ☐ Both: Not exposed to 0.0.0.0 (bind to localhost or VPN)

- ☐ Both: Access restricted via firewall
- ☐ Both: Connection strings stored in secrets manager

Risk #4: Missing TLS/HTTPS

Severity: 🟠 High
Common in: Internal services, development environments

The Problem

HTTP (unencrypted) exposes all traffic to interception. Credentials, tokens, and data are sent in plaintext.

Real-World Impact

- **2025:** Internal API over HTTP — attacker on same network captured admin tokens.
- **2024:** Development dashboard over HTTP — credentials stolen via ARP spoofing.

The Fix

```
# Use Let's Encrypt (free, automated)
certbot --nginx -d your-domain.com

# Or self-signed cert for internal services
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/server.key \
  -out /etc/ssl/certs/server.crt

# Enforce HTTPS (Nginx example)
server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$host$request_uri;
}
```

Prevention Checklist

- ☐ All public services use HTTPS
- ☐ Internal services use HTTPS or VPN
- ☐ HSTS enabled
- ☐ Certificates auto-renewed
- ☐ Weak ciphers disabled (TLS 1.2+ only)

Risk #5: Exposed Admin Panels

Severity: 🟠 High
Common in: Portainer, Grafana, Jenkins, WordPress

The Problem

Admin panels with default credentials or no authentication are a common attack vector.

Real-World Impact

- **2025:** Portainer with default admin/admin — attacker deployed malicious containers.
- **2024:** Grafana exposed without auth — attacker accessed all dashboards.

The Fix

```
# 1. Change default credentials immediately
# 2. Enable 2FA if available
# 3. Restrict access via IP whitelist
# 4. Use reverse proxy with authentication (OAuth, SSO)

# Nginx auth_basic example:
location /admin {
    auth_basic "Restricted";
    auth_basic_user_file /etc/nginx/.htpasswd;
}
```

Prevention Checklist

- ☐ Default credentials changed
- ☐ 2FA enabled where possible
- ☐ Access restricted to trusted IPs
- ☐ Admin panels not exposed to 0.0.0.0
- ☐ Audit logs enabled

Risk #6: Unrestricted Docker Registry

Severity: □ High
Common in: CI/CD pipelines, development workflows

The Problem

If your Docker registry (Harbor, Nexus, GitLab Registry) allows anonymous pulls or has weak auth, attackers can: - Pull your proprietary images - Push malicious images - Supply chain attacks

The Fix

```
# Harbor: Disable anonymous access
# System Settings -> Authentication -> Disable "Anonymous"

# GitLab Registry: Require auth for pulls
# Settings -> Registry -> Access Control -> "Everyone" -> "Only authenticated users"

# Use image signing (cosign)
cosign sign your-registry/your-image:tag
cosign verify your-registry/your-image:tag
```

Prevention Checklist

- ☐ Anonymous access disabled
- ☐ Strong authentication (RBAC)
- ☐ Image scanning enabled (Trivy, Clair)
- ☐ Image signing enforced
- ☐ Pull/push logs monitored

Risk #7: Outdated Dependencies

Severity: Medium
Common in: All self-hosted applications

The Problem

Outdated libraries often contain known CVEs. Attackers scan for vulnerable versions.

Real-World Impact

- **2025:** Log4j vulnerability still found in legacy apps.
- **2024:** Spring4Shell exploit in outdated Spring Boot.

The Fix

```
# Automated dependency updates
npm audit fix
pip install --upgrade
cargo update

# Container scanning
trivy image your-image:tag

# SBOM generation
syft your-image:tag -o spdx-json > sbom.json
```

Prevention Checklist

- ☐ Automated dependency scanning (Renovate, Dependabot)
- ☐ Container images scanned before deployment
- ☐ SBOM generated for all images
- ☐ CVE alerts monitored
- ☐ Patching schedule (weekly)

Risk #8: Missing Firewall Rules

Severity: Medium
Common in: Cloud instances, homelabs

The Problem

Open firewall ports allow unauthorized access. Common mistake: exposing SSH (22) or database ports to the world.

The Fix

```
# UFW (Ubuntu/Debian)
ufw default deny incoming
ufw default allow outgoing
ufw allow from 192.168.1.0/24 to any port 22
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable

# iptables (manual)
iptables -A INPUT -p tcp --dport 22 -s 192.168.1.0/24 -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j DROP
```

Prevention Checklist

- ☐ Default deny policy
- ☐ Only necessary ports open
- ☐ SSH restricted to trusted IPs
- ☐ Database ports not exposed to internet
- ☐ Firewall rules documented

Risk #9: No Backup Strategy

Severity: □ Medium
Common in: Small deployments, development environments

The Problem

No backups = data loss on hardware failure, ransomware, or accidental deletion.

The Fix

```
# Automated backups (restic example)
restic backup /data --repo s3:backup-bucket
restic forget --keep-daily 7 --keep-weekly 4

# Database backups (PostgreSQL example)
pg_dump -U user dbname > backup.sql
# Automate with cron:
0 2 * * * pg_dump -U user dbname | gzip > /backups/db-$(date +\%Y\%m\%d).sql.gz
```

Prevention Checklist

- ☐ Automated daily backups
- ☐ Backups stored off-site
- ☐ Backups encrypted

- ☐ Restore tested quarterly
- ☐ Retention policy (7 daily, 4 weekly)

Risk #10: Insufficient Logging/Monitoring

Severity: Medium
Common in: All self-hosted infrastructure

The Problem

No logs = no visibility. You can't detect attacks if you don't know what's happening.

The Fix

```
# Centralized logging (Loki/ELK)
# Forward logs to Loki:
loki -> promtail -> grafana

# Alerting (Prometheus Alertmanager)
# Alert on:
# - High CPU/memory
# - Failed SSH attempts
# - Unusual traffic patterns
# - Service downtime
```

Prevention Checklist

- ☐ Centralized logging (Loki, ELK)
- ☐ Metrics collection (Prometheus)
- ☐ Alerting configured (Alertmanager)
- ☐ Logs retained 30+ days
- ☐ Security events monitored

Quick Reference Checklist

Print this checklist and review monthly:

- ☐ Docker socket not exposed
- ☐ SSH keys unique per server
- ☐ Redis/MongoDB authenticated
- ☐ HTTPS everywhere
- ☐ Admin panels protected
- ☐ Docker registry secured
- ☐ Dependencies updated
- ☐ Firewall rules strict
- ☐ Backups automated
- ☐ Logging/monitoring enabled

Next Steps

1. **Run a Security Check**
   Use ClawGuru Security Check to scan your stack for these risks automatically.
2. **Implement Fixes**
   Start with Critical risks (1-3), then High (4-6), then Medium (7-10).
3. **Set Up Monitoring**
   Deploy alerts for failed logins, unusual traffic, and service downtime.
4. **Review Quarterly**
   Security is not one-time. Review this checklist every quarter.

About ClawGuru

ClawGuru is the #1 security check platform for self-hosted infrastructure. We scan your stack, identify risks, and provide actionable fixes.

Free forever for individuals.
Pro for teams: Unlimited scans, runbooks, priority support.

Get Started Free | View Pricing

Version: 1.0
Last Updated: April 2026

> "Security is not a product, but a process." — Bruce Schneier